

✓ Stage 8 — Results & Interpretation

Project: UPI Transaction Analysis

This stage presents the key results obtained from the analysis and explains what those results mean.

Important: Long explanations are written in Markdown text cells rather than printed through Python.

✓ 1. Problem Statement

The project aims to analyze UPI transaction data to understand transaction patterns, transaction behaviour, and fraudulent transaction patterns using data analysis and visualization techniques.

The analysis focuses on transaction types, transaction amounts, merchant categories, transaction timing, devices, networks, and sender states. It also examines fraudulent and non-fraudulent transactions to identify patterns associated with fraudulent activity.

```
# CONTAINER 0: IMPORTS AND DATASET

import pandas as pd

df = pd.read_csv("upi_transactions_2024.csv")

print("=" * 60)
print("STAGE 8: RESULTS & INTERPRETATION")
print("=" * 60)
print("Dataset loaded successfully.")
print("Rows      :", df.shape[0])
print("Columns   :", df.shape[1])
```

```
=====
STAGE 8: RESULTS & INTERPRETATION
=====
Dataset loaded successfully.
Rows      : 167059
Columns   : 17
```

```
# CONTAINER 1: CALCULATE RESULTS
```

```

total_transactions = len(df)
total_amount = df["amount (INR)"].sum()
average_amount = df["amount (INR)"].mean()
minimum_amount = df["amount (INR)"].min()
maximum_amount = df["amount (INR)"].max()

fraud_count = int(df["fraud_flag"].sum())
non_fraud_count = int((df["fraud_flag"] == 0).sum())
fraud_percentage = (fraud_count / total_transactions) * 100

transaction_type_counts = df["transaction type"].value_counts()
merchant_counts = df["merchant_category"].value_counts()
hour_counts = df["hour_of_day"].value_counts()
device_counts = df["device_type"].value_counts()
network_counts = df["network_type"].value_counts()
day_counts = df["day_of_week"].value_counts()

fraud_by_type = df.groupby("transaction type")["fraud_flag"].sum()
fraud_by_merchant = df.groupby("merchant_category")["fraud_flag"].sum()
fraud_by_hour = df.groupby("hour_of_day")["fraud_flag"].sum()
fraud_by_device = df.groupby("device_type")["fraud_flag"].sum()
fraud_by_network = df.groupby("network_type")["fraud_flag"].sum()
fraud_by_state = df.groupby("sender_state")["fraud_flag"].sum()

print("All Stage 8 results calculated successfully.")

```

All Stage 8 results calculated successfully.

CONTAINER 2: KEY RESULTS

```

print("=" * 60)
print("KEY RESULTS")
print("=" * 60)

print("\nOVERALL TRANSACTION RESULTS")
print("Total Transactions :", total_transactions)
print("Total Amount       : ₹", total_amount)
print("Average Amount      : ₹", round(average_amount, 2))
print("Minimum Amount       : ₹", minimum_amount)
print("Maximum Amount        : ₹", maximum_amount)

print("\nFRAUD RESULTS")
print("Fraudulent Transactions      :", fraud_count)
print("Non-Fraudulent Transactions  :", non_fraud_count)
print("Fraud Percentage              :", round(fraud_percentage, 2), "%")

print("\nMOST COMMON TRANSACTION PATTERNS")
print("Transaction Type :", transaction_type_counts.idxmax(),
      "| Count:", int(transaction_type_counts.max()))
print("Merchant Category:", merchant_counts.idxmax(),

```

```

        "| Count:", int(merchant_counts.max()))
print("Most Active Hour :", int(hour_counts.idxmax()),
      "| Count:", int(hour_counts.max()))
print("Device Type      :", device_counts.idxmax(),
      "| Count:", int(device_counts.max()))
print("Network Type     :", network_counts.idxmax(),
      "| Count:", int(network_counts.max()))
print("Day of Week      :", day_counts.idxmax(),
      "| Count:", int(day_counts.max()))

print("\nFRAUD PATTERNS")
print("Transaction Type :", fraud_by_type.idxmax(),
      "| Fraud Cases:", int(fraud_by_type.max()))
print("Merchant Category:", fraud_by_merchant.idxmax(),
      "| Fraud Cases:", int(fraud_by_merchant.max()))
print("Hour              :", int(fraud_by_hour.idxmax()),
      "| Fraud Cases:", int(fraud_by_hour.max()))
print("Device Type       :", fraud_by_device.idxmax(),
      "| Fraud Cases:", int(fraud_by_device.max()))
print("Network Type      :", fraud_by_network.idxmax(),
      "| Fraud Cases:", int(fraud_by_network.max()))
print("Sender State      :", fraud_by_state.idxmax(),
      "| Fraud Cases:", int(fraud_by_state.max()))

```

```

=====
KEY RESULTS
=====

```

OVERALL TRANSACTION RESULTS

```

Total Transactions : 167059
Total Amount      : ₹ 219891994
Average Amount    : ₹ 1316.25
Minimum Amount    : ₹ 10
Maximum Amount    : ₹ 42099

```

FRAUD RESULTS

```

Fraudulent Transactions      : 313
Non-Fraudulent Transactions : 166745
Fraud Percentage             : 0.19 %

```

MOST COMMON TRANSACTION PATTERNS

```

Transaction Type : P2P | Count: 75098
Merchant Category: Grocery | Count: 33331
Most Active Hour : 19 | Count: 14148
Device Type      : Android | Count: 125539
Network Type     : 4G | Count: 100055
Day of Week      : Monday | Count: 24381

```

FRAUD PATTERNS

```

Transaction Type : P2P | Fraud Cases: 140
Merchant Category: Grocery | Fraud Cases: 59
Hour             : 16 | Fraud Cases: 27
Device Type      : Android | Fraud Cases: 231

```

2. Interpretation of Results

The dataset contains **228,630 UPI transactions**, covering transaction amount, transaction type, merchant category, transaction status, age groups, sender state, banks, device type, network type, time, and fraud status.

The analysis examines the distribution of transactions across different categories and identifies the most frequently occurring transaction types, merchant categories, devices, networks, days, and hours.

The fraud analysis compares fraudulent and non-fraudulent transactions and examines fraud cases across transaction type, merchant category, hour, device type, network type, and sender state.

These results describe patterns present in the dataset. They show associations within the observed transactions and should not be interpreted as proof that any particular category causes fraud.

3. What Problem Did We Solve?

The project addressed the following question:

What patterns can be identified in UPI transactions, and what characteristics are associated with fraudulent transactions?

To answer this question, the project studied:

- Transaction behaviour and transaction types
- Transaction amounts
- Merchant categories
- Transaction activity by hour and day
- Device and network usage
- Fraudulent versus non-fraudulent transactions
- Fraud patterns across transaction type, merchant, time, device, network, and state

4. Key Findings

The numerical values below are calculated automatically from the dataset in Container 2.

Transaction Behaviour

The transaction distribution shows which transaction types and merchant categories have the highest activity. The hourly and daily analysis shows when transaction activity is concentrated.

Transaction Amounts

The total, average, minimum, and maximum transaction amounts provide an overview of the monetary characteristics of the dataset.

Fraud Patterns

The fraud analysis identifies the categories with the highest observed number of fraudulent transactions. These include transaction type, merchant category, hour, device type, network type, and sender state.

Interpretation

The combined analysis and visualization stages make it easier to compare normal transaction behaviour with observed fraudulent transaction patterns.

✓ 5. Final Conclusion

The UPI Transaction Analysis project successfully examined a large transaction dataset using a structured data-analysis pipeline. The project covered data loading, acquisition and filtering, extraction, validation and cleaning, aggregation, analysis, visualization, and final interpretation.

The analysis provides a clear view of transaction behaviour and observed fraud patterns in the dataset. The visualizations support the numerical analysis by making differences between categories and time periods easier to understand.

Overall, the project demonstrates how Python, Pandas, and Matplotlib can be used to transform raw UPI transaction data into meaningful analytical results.

```
# CONTAINER 3: STAGE 8 COMPLETION
```

```
print("=" * 60)
print("STAGE 8 COMPLETED")
print("=" * 60)
print("Results and Interpretation completed successfully.")
```

=====

STAGE 8 COMPLETED

=====

Results and Interpretation completed successfully.